Contents

DSA Solutions Guide - Part 4: Dynamic Programming & Backtracking **3**
Complete JavaScript Solutions with Detailed Explanations . . . 3

DYNAMIC PROGRAMMING **3**
Understanding DP . . . 3

1. Climbing Stairs **3**
Problem . . . 3
Example . . . 3
Approach: Fibonacci Pattern . . . 3
Why It Works . . . 4

2. House Robber **4**
Problem . . . 4
Example . . . 4
Approach . . . 4

3. House Robber II (Circular) **5**
Problem . . . 5

4. Coin Change **6**
Problem . . . 6
Example . . . 6
Approach: Bottom-Up DP . . . 6
Visual Walkthrough . . . 6

5. Coin Change 2 (Count Ways) **6**
Problem . . . 6
Why Loop Order Matters . . . 7

6. Longest Increasing Subsequence **7**
Problem . . . 7
Example . . . 7
Approach 1: DP - O(n^2) . . . 7
Approach 2: Binary Search - O(n log n) . . . 7
Visual Walkthrough . . . 8

7. Longest Common Subsequence **8**
Problem . . . 8
Example . . . 8
Approach: 2D DP . . . 8

8. Edit Distance **9**
Problem . . . 9
Example . . . 9
Approach: 2D DP . . . 10

9. Word Break **10**
Problem . . . 10
Example . . . 10
Approach: DP . . . 10

10. Unique Paths **11**
Problem . . . 11

11. Minimum Path Sum **12**

Problem . . . 12

12. Maximal Square **12**
Problem . . . 12
Why min of three neighbors? . . . 13

13. 0/1 Knapsack **13**
Problem . . . 13

14. Partition Equal Subset Sum **14**
Problem . . . 14

15. Decode Ways **14**
Problem . . . 14

BACKTRACKING **15**
Understanding Backtracking . . . 15

16. Subsets **15**
Problem . . . 15
Example . . . 15

17. Subsets II (with Duplicates) **16**
Problem . . . 16

18. Permutations **17**
Problem . . . 17

19. Permutations II (with Duplicates) **18**

20. Combination Sum **18**
Problem . . . 18
Example . . . 18

21. Combination Sum II (No Reuse) **19**

22. Generate Parentheses **19**
Problem . . . 19
Example . . . 20

23. Letter Combinations of Phone Number **20**
Problem . . . 20

24. Word Search **21**
Problem . . . 21

25. N-Queens **22**
Problem . . . 22

26. Sudoku Solver **22**

Summary: DP & Backtracking Patterns **23**
DP Patterns . . . 23
Backtracking Patterns . . . 24
Key Differences . . . 24

DSA Solutions Guide - Part 4: Dynamic Programming & Backtracking

Complete JavaScript Solutions with Detailed Explanations

DYNAMIC PROGRAMMING

Understanding DP

When to use DP: 1. **Optimal Substructure:** Solution can be built from solutions to subproblems 2. **Overlapping Subproblems:** Same subproblems solved multiple times

Two Approaches: - **Top-Down (Memoization):** Start from main problem, cache results - **Bottom-Up (Tabulation):** Build solution from smallest subproblems

---

1. Climbing Stairs

Problem

You can climb 1 or 2 steps. How many ways to reach step n?

Example

```
n = 3 -> 3 ways: (1+1+1), (1+2), (2+1)
n = 4 -> 5 ways
```

Approach: Fibonacci Pattern

```javascript
// Top-Down with Memoization
function climbStairsMemo(n, memo = {}) {
    if (n <= 2) return n;
    if (memo[n]) return memo[n];

    memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);
    return memo[n];
}

// Bottom-Up with Tabulation
function climbStairsTab(n) {
    if (n <= 2) return n;

    const dp = new Array(n + 1);
    dp[1] = 1;
    dp[2] = 2;

    for (let i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}

// Space Optimized - O(1)
function climbStairs(n) {
    if (n <= 2) return n;
```

```
    let prev2 = 1;  // ways to reach step 1
    let prev1 = 2;  // ways to reach step 2

    for (let i = 3; i <= n; i++) {
        const current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}

// Time: O(n), Space: O(1)
```

Why It Works

To reach step n, you either: - Come from step n-1 (1 step) - Come from step n-2 (2 steps)

So: `ways(n) = ways(n-1) + ways(n-2)`

---

2. House Robber

Problem

Rob houses with maximum money without robbing adjacent houses.

Example

```
nums = [1, 2, 3, 1] -> 4 (rob house 0 and 2)
nums = [2, 7, 9, 3, 1] -> 12 (rob house 0, 2, 4)
```

Approach

At each house, choose: rob it (skip previous) or skip it (take previous best).

```
// Bottom-Up
function rob(nums) {
    if (nums.length === 0) return 0;
    if (nums.length === 1) return nums[0];

    const dp = new Array(nums.length);
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (let i = 2; i < nums.length; i++) {
        dp[i] = Math.max(
            dp[i - 1],              // Skip current house
            dp[i - 2] + nums[i]     // Rob current house
        );
    }

    return dp[nums.length - 1];
}

// Space Optimized
```

```
function robOptimized(nums) {
    if (nums.length === 0) return 0;
    if (nums.length === 1) return nums[0];

    let prev2 = 0;  // Max if we're 2 houses back
    let prev1 = 0;  // Max if we're 1 house back

    for (const num of nums) {
        const current = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}

// Time: O(n), Space: O(1)
```

---

3. House Robber II (Circular)

Problem

Houses are arranged in a circle (first and last are adjacent).

```
function robCircular(nums) {
    if (nums.length === 0) return 0;
    if (nums.length === 1) return nums[0];
    if (nums.length === 2) return Math.max(nums[0], nums[1]);

    // Rob houses 0 to n-2 OR houses 1 to n-1
    return Math.max(
        robRange(nums, 0, nums.length - 2),
        robRange(nums, 1, nums.length - 1)
    );
}

function robRange(nums, start, end) {
    let prev2 = 0;
    let prev1 = 0;

    for (let i = start; i <= end; i++) {
        const current = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}

// Time: O(n), Space: O(1)
```

---

4. Coin Change

Problem

Find minimum coins needed to make amount. Return -1 if impossible.

Example

```
coins = [1, 2, 5], amount = 11 -> 3 (5 + 5 + 1)
coins = [2], amount = 3 -> -1
```

Approach: Bottom-Up DP

`dp[i]` = minimum coins to make amount i

```javascript
function coinChange(coins, amount) {
    // dp[i] = min coins to make amount i
    const dp = new Array(amount + 1).fill(Infinity);
    dp[0] = 0;  // 0 coins needed for amount 0

    for (let i = 1; i <= amount; i++) {
        for (const coin of coins) {
            if (coin <= i && dp[i - coin] !== Infinity) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] === Infinity ? -1 : dp[amount];
}

// Time: O(amount * coins), Space: O(amount)
```

Visual Walkthrough

```
coins = [1, 2, 5], amount = 11

dp[0] = 0
dp[1] = min(dp[0]+1) = 1         (use coin 1)
dp[2] = min(dp[1]+1, dp[0]+1) = 1 (use coin 2)
dp[3] = min(dp[2]+1, dp[1]+1) = 2 (1+2 or 1+1+1)
dp[4] = min(dp[3]+1, dp[2]+1) = 2 (2+2)
dp[5] = min(dp[4]+1, dp[3]+1, dp[0]+1) = 1 (use coin 5)
...
dp[11] = 3 (5+5+1)
```

---

5. Coin Change 2 (Count Ways)

Problem

Count number of combinations to make amount.

```javascript
function change(amount, coins) {
    const dp = new Array(amount + 1).fill(0);
    dp[0] = 1;  // One way to make 0: use no coins

    // For each coin, update all amounts it can contribute to
```

```
    for (const coin of coins) {
        for (let i = coin; i <= amount; i++) {
            dp[i] += dp[i - coin];
        }
    }

    return dp[amount];
}

// Time: O(amount * coins), Space: O(amount)
```

Why Loop Order Matters

- Loop coins outside: each coin considered once per combination (combinations)
- Loop amounts outside: same coin can be used multiple times in sequence (permutations)

---

6. Longest Increasing Subsequence

Problem

Find the length of the longest strictly increasing subsequence.

Example

```
nums = [10, 9, 2, 5, 3, 7, 101, 18] -> 4 ([2, 3, 7, 101])
```

Approach 1: DP - O(n^2)

```
function lengthOfLIS_DP(nums) {
    if (nums.length === 0) return 0;

    // dp[i] = length of LIS ending at index i
    const dp = new Array(nums.length).fill(1);

    for (let i = 1; i < nums.length; i++) {
        for (let j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
    }

    return Math.max(...dp);
}

// Time: O(n^2), Space: O(n)
```

Approach 2: Binary Search - O(n log n)

Maintain array of smallest tail elements for each LIS length.

```
function lengthOfLIS(nums) {
    // tails[i] = smallest tail element for LIS of length i+1
    const tails = [];

    for (const num of nums) {
```

```
        // Binary search for position to insert/replace
        let left = 0;
        let right = tails.length;

        while (left < right) {
            const mid = Math.floor((left + right) / 2);
            if (tails[mid] < num) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        if (left === tails.length) {
            tails.push(num);  // Extend LIS
        } else {
            tails[left] = num;  // Replace with smaller value
        }
    }

    return tails.length;
}

// Time: O(n log n), Space: O(n)
```

Visual Walkthrough

```
nums = [10, 9, 2, 5, 3, 7, 101, 18]

num=10: tails = [10]
num=9:  tails = [9]        (replace 10)
num=2:  tails = [2]        (replace 9)
num=5:  tails = [2, 5]     (extend)
num=3:  tails = [2, 3]     (replace 5)
num=7:  tails = [2, 3, 7]  (extend)
num=101: tails = [2, 3, 7, 101] (extend)
num=18: tails = [2, 3, 7, 18]   (replace 101)

Length = 4
```

---

7. Longest Common Subsequence

Problem

Find the length of the longest common subsequence of two strings.

Example

```
text1 = "abcde", text2 = "ace" -> 3 ("ace")
```

Approach: 2D DP

```
function longestCommonSubsequence(text1, text2) {
    const m = text1.length;
    const n = text2.length;
```

```
    // dp[i][j] = LCS of text1[0..i-1] and text2[0..j-1]
    const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            if (text1[i - 1] === text2[j - 1]) {
                // Characters match: extend LCS
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // Take best of excluding one character
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[m][n];
}

// Space Optimized - O(min(m, n))
function longestCommonSubsequenceOptimized(text1, text2) {
    if (text1.length < text2.length) {
        [text1, text2] = [text2, text1];
    }

    const n = text2.length;
    let prev = new Array(n + 1).fill(0);
    let curr = new Array(n + 1).fill(0);

    for (let i = 1; i <= text1.length; i++) {
        for (let j = 1; j <= n; j++) {
            if (text1[i - 1] === text2[j - 1]) {
                curr[j] = prev[j - 1] + 1;
            } else {
                curr[j] = Math.max(prev[j], curr[j - 1]);
            }
        }
        [prev, curr] = [curr, prev];
    }

    return prev[n];
}

// Time: O(m * n), Space: O(min(m, n))
```

---

8. Edit Distance

Problem

Minimum operations (insert, delete, replace) to convert word1 to word2.

Example

```
word1 = "horse", word2 = "ros" -> 3
  horse -> rorse (replace h with r)
```

```
  rorse -> rose (remove r)
  rose -> ros (remove e)
```

Approach: 2D DP

```
function minDistance(word1, word2) {
    const m = word1.length;
    const n = word2.length;

    // dp[i][j] = min operations to convert word1[0..i-1] to word2[0..j-1]
    const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

    // Base cases: converting to/from empty string
    for (let i = 0; i <= m; i++) dp[i][0] = i;  // Delete all
    for (let j = 0; j <= n; j++) dp[0][j] = j;  // Insert all

    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            if (word1[i - 1] === word2[j - 1]) {
                // Characters match: no operation needed
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(
                    dp[i - 1][j - 1],  // Replace
                    dp[i - 1][j],      // Delete from word1
                    dp[i][j - 1]       // Insert into word1
                );
            }
        }
    }

    return dp[m][n];
}

// Time: O(m * n), Space: O(m * n)
```

---

9. Word Break

Problem

Determine if string can be segmented into dictionary words.

Example

```
s = "leetcode", wordDict = ["leet", "code"] -> true
s = "applepenapple", wordDict = ["apple", "pen"] -> true
```

Approach: DP

`dp[i]` = true if s[0..i-1] can be segmented

```
function wordBreak(s, wordDict) {
    const wordSet = new Set(wordDict);
    const n = s.length;

    // dp[i] = true if s[0..i-1] can be segmented
```

```
    const dp = new Array(n + 1).fill(false);
    dp[0] = true;  // Empty string

    for (let i = 1; i <= n; i++) {
        for (let j = 0; j < i; j++) {
            // If s[0..j-1] can be segmented AND s[j..i-1] is a word
            if (dp[j] && wordSet.has(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[n];
}

// Time: O(n^2 * k) where k = avg word length
// Space: O(n)
```

---

10. Unique Paths

Problem

Count unique paths from top-left to bottom-right in a grid (can only move right or down).

```
function uniquePaths(m, n) {
    // dp[i][j] = number of ways to reach cell (i, j)
    const dp = Array.from({ length: m }, () => new Array(n).fill(1));

    // First row and column are all 1s (only one way to reach)

    for (let i = 1; i < m; i++) {
        for (let j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }

    return dp[m - 1][n - 1];
}

// Space Optimized
function uniquePathsOptimized(m, n) {
    const dp = new Array(n).fill(1);

    for (let i = 1; i < m; i++) {
        for (let j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }

    return dp[n - 1];
}

// Time: O(m * n), Space: O(n)
```

---

11. Minimum Path Sum

Problem

Find minimum path sum from top-left to bottom-right.

```
function minPathSum(grid) {
    const m = grid.length;
    const n = grid[0].length;

    // Modify in place (or use separate dp array)
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (i === 0 && j === 0) continue;

            const fromTop = i > 0 ? grid[i - 1][j] : Infinity;
            const fromLeft = j > 0 ? grid[i][j - 1] : Infinity;

            grid[i][j] += Math.min(fromTop, fromLeft);
        }
    }

    return grid[m - 1][n - 1];
}

// Time: O(m * n), Space: O(1) if modifying in place
```

---

12. Maximal Square

Problem

Find the largest square containing only 1s in a binary matrix.

```
function maximalSquare(matrix) {
    if (matrix.length === 0) return 0;

    const m = matrix.length;
    const n = matrix[0].length;

    // dp[i][j] = side length of largest square with bottom-right at (i, j)
    const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
    let maxSide = 0;

    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            if (matrix[i - 1][j - 1] === '1') {
                // Minimum of three neighbors + 1
                dp[i][j] = 1 + Math.min(
                    dp[i - 1][j],      // Top
                    dp[i][j - 1],      // Left
                    dp[i - 1][j - 1]   // Top-left
                );
                maxSide = Math.max(maxSide, dp[i][j]);
            }
        }
    }
```

```
    return maxSide * maxSide;
}

// Time: O(m * n), Space: O(m * n)
```

Why min of three neighbors?

If any neighbor has a smaller square, that's the limiting factor for extending.

---

13. 0/1 Knapsack

Problem

Given items with weights and values, maximize value within weight capacity.

```
function knapsack(weights, values, capacity) {
    const n = weights.length;

    // dp[i][w] = max value using first i items with capacity w
    const dp = Array.from({ length: n + 1 }, () => new Array(capacity + 1).fill(0));

    for (let i = 1; i <= n; i++) {
        for (let w = 0; w <= capacity; w++) {
            // Don't take item i
            dp[i][w] = dp[i - 1][w];

            // Take item i (if it fits)
            if (weights[i - 1] <= w) {
                dp[i][w] = Math.max(
                    dp[i][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]
                );
            }
        }
    }

    return dp[n][capacity];
}

// Space Optimized - O(capacity)
function knapsackOptimized(weights, values, capacity) {
    const dp = new Array(capacity + 1).fill(0);

    for (let i = 0; i < weights.length; i++) {
        // Traverse backwards to avoid using same item twice
        for (let w = capacity; w >= weights[i]; w--) {
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}

// Time: O(n * capacity), Space: O(capacity)
```

---

14. Partition Equal Subset Sum

Problem

Can array be partitioned into two subsets with equal sum?

```
function canPartition(nums) {
    const totalSum = nums.reduce((a, b) => a + b, 0);

    // If sum is odd, can't partition equally
    if (totalSum % 2 !== 0) return false;

    const target = totalSum / 2;

    // dp[i] = true if sum i is achievable
    const dp = new Array(target + 1).fill(false);
    dp[0] = true;

    for (const num of nums) {
        // Traverse backwards to avoid reusing same number
        for (let j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }

    return dp[target];
}

// Time: O(n * sum), Space: O(sum)
```

---

15. Decode Ways

Problem

Count ways to decode a numeric string where 'A' = 1, 'B' = 2, . . . , 'Z' = 26.

```
function numDecodings(s) {
    if (s.length === 0 || s[0] === '0') return 0;

    const n = s.length;

    // dp[i] = ways to decode s[0..i-1]
    const dp = new Array(n + 1).fill(0);
    dp[0] = 1;  // Empty string
    dp[1] = s[0] !== '0' ? 1 : 0;

    for (let i = 2; i <= n; i++) {
        // Single digit decode
        const oneDigit = parseInt(s[i - 1]);
        if (oneDigit >= 1 && oneDigit <= 9) {
            dp[i] += dp[i - 1];
        }
```

```
        // Two digit decode
        const twoDigit = parseInt(s.substring(i - 2, i));
        if (twoDigit >= 10 && twoDigit <= 26) {
            dp[i] += dp[i - 2];
        }
    }

    return dp[n];
}

// Time: O(n), Space: O(n) or O(1) with optimization
```

BACKTRACKING

Understanding Backtracking

Backtracking is a systematic way to explore all possible configurations. It builds solutions incrementally, abandoning a path as soon as it determines that path cannot lead to a valid solution.

Template:

```
function backtrack(state) {
    if (isGoal(state)) {
        recordSolution(state);
        return;
    }

    for (choice of getChoices(state)) {
        if (isValid(choice)) {
            makeChoice(state, choice);
            backtrack(state);
            undoChoice(state, choice);  // Backtrack
        }
    }
}
```

---

16. Subsets

Problem

Return all possible subsets of an array.

Example

```
nums = [1, 2, 3]
Output: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

```
function subsets(nums) {
    const result = [];

    function backtrack(start, current) {
        // Add current subset (make a copy)
        result.push([...current]);
```

```
        // Try adding each remaining element
        for (let i = start; i < nums.length; i++) {
            current.push(nums[i]);       // Choose
            backtrack(i + 1, current);   // Explore
            current.pop();               // Unchoose (backtrack)
        }
    }

    backtrack(0, []);
    return result;
}

// Iterative approach
function subsetsIterative(nums) {
    const result = [[]];

    for (const num of nums) {
        const newSubsets = result.map(subset => [...subset, num]);
        result.push(...newSubsets);
    }

    return result;
}

// Time: O(n * 2^n), Space: O(n) for recursion
```

---

17. Subsets II (with Duplicates)

Problem

Return all unique subsets when array may contain duplicates.

```
function subsetsWithDup(nums) {
    const result = [];
    nums.sort((a, b) => a - b);  // Sort to handle duplicates

    function backtrack(start, current) {
        result.push([...current]);

        for (let i = start; i < nums.length; i++) {
            // Skip duplicates at same level
            if (i > start && nums[i] === nums[i - 1]) continue;

            current.push(nums[i]);
            backtrack(i + 1, current);
            current.pop();
        }
    }

    backtrack(0, []);
    return result;
}

// Time: O(n * 2^n), Space: O(n)
```

---

18. Permutations

Problem

Return all possible permutations of an array.

```
function permute(nums) {
    const result = [];

    function backtrack(current, remaining) {
        if (remaining.length === 0) {
            result.push([...current]);
            return;
        }

        for (let i = 0; i < remaining.length; i++) {
            current.push(remaining[i]);

            // Create new remaining array without current element
            const newRemaining = [...remaining.slice(0, i), ...remaining.slice(i + 1)];

            backtrack(current, newRemaining);
            current.pop();
        }
    }

    backtrack([], nums);
    return result;
}

// Using swap (in-place)
function permuteSwap(nums) {
    const result = [];

    function backtrack(start) {
        if (start === nums.length) {
            result.push([...nums]);
            return;
        }

        for (let i = start; i < nums.length; i++) {
            [nums[start], nums[i]] = [nums[i], nums[start]];  // Swap
            backtrack(start + 1);
            [nums[start], nums[i]] = [nums[i], nums[start]];  // Swap back
        }
    }

    backtrack(0);
    return result;
}

// Time: O(n! * n), Space: O(n)
```

---

19. Permutations II (with Duplicates)

```
function permuteUnique(nums) {
    const result = [];
    nums.sort((a, b) => a - b);
    const used = new Array(nums.length).fill(false);

    function backtrack(current) {
        if (current.length === nums.length) {
            result.push([...current]);
            return;
        }

        for (let i = 0; i < nums.length; i++) {
            // Skip used elements
            if (used[i]) continue;

            // Skip duplicates: if previous same element not used, skip this
            if (i > 0 && nums[i] === nums[i - 1] && !used[i - 1]) continue;

            used[i] = true;
            current.push(nums[i]);
            backtrack(current);
            current.pop();
            used[i] = false;
        }
    }

    backtrack([]);
    return result;
}

// Time: O(n! * n), Space: O(n)
```

---

20. Combination Sum

Problem

Find all combinations that sum to target (can reuse elements).

Example

```
candidates = [2, 3, 6, 7], target = 7
Output: [[2, 2, 3], [7]]
```

```
function combinationSum(candidates, target) {
    const result = [];

    function backtrack(start, current, remaining) {
        if (remaining === 0) {
            result.push([...current]);
            return;
        }

        if (remaining < 0) return;
```

```
        for (let i = start; i < candidates.length; i++) {
            current.push(candidates[i]);
            // Use i (not i+1) to allow reuse
            backtrack(i, current, remaining - candidates[i]);
            current.pop();
        }
    }

    backtrack(0, [], target);
    return result;
}

// Time: O(n^(target/min)), Space: O(target/min)
```

---

21. Combination Sum II (No Reuse)

```
function combinationSum2(candidates, target) {
    const result = [];
    candidates.sort((a, b) => a - b);

    function backtrack(start, current, remaining) {
        if (remaining === 0) {
            result.push([...current]);
            return;
        }

        if (remaining < 0) return;

        for (let i = start; i < candidates.length; i++) {
            // Skip duplicates at same level
            if (i > start && candidates[i] === candidates[i - 1]) continue;

            // Pruning: if current element > remaining, skip rest
            if (candidates[i] > remaining) break;

            current.push(candidates[i]);
            backtrack(i + 1, current, remaining - candidates[i]);  // i+1 for no reuse
            current.pop();
        }
    }

    backtrack(0, [], target);
    return result;
}

// Time: O(2^n), Space: O(n)
```

---

22. Generate Parentheses

Problem

Generate all valid combinations of n pairs of parentheses.

Example

```
n = 3 -> ["((()))", "(()())", "(())()", "()(())", "()()()"]
```

```
function generateParenthesis(n) {
    const result = [];

    function backtrack(current, open, close) {
        if (current.length === 2 * n) {
            result.push(current);
            return;
        }

        // Can add open paren if we have some left
        if (open < n) {
            backtrack(current + '(', open + 1, close);
        }

        // Can add close paren if it won't exceed open
        if (close < open) {
            backtrack(current + ')', open, close + 1);
        }
    }

    backtrack('', 0, 0);
    return result;
}

// Time: O(4^n / sqrt(n)), Space: O(n)
```

---

23. Letter Combinations of Phone Number

Problem

Return all letter combinations that the number could represent.

```
function letterCombinations(digits) {
    if (digits.length === 0) return [];

    const mapping = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    };

    const result = [];

    function backtrack(index, current) {
        if (index === digits.length) {
            result.push(current);
            return;
        }

        const letters = mapping[digits[index]];
        for (const letter of letters) {
            backtrack(index + 1, current + letter);
        }
```

```
    }

    backtrack(0, '');
    return result;
}

// Time: O(4^n * n), Space: O(n)
```

---

24. Word Search

Problem

Find if a word exists in a 2D grid (can move up, down, left, right).

```
function exist(board, word) {
    const rows = board.length;
    const cols = board[0].length;

    function backtrack(r, c, index) {
        // Found complete word
        if (index === word.length) return true;

        // Boundary and character check
        if (r < 0 || r >= rows || c < 0 || c >= cols) return false;
        if (board[r][c] !== word[index]) return false;

        // Mark as visited
        const temp = board[r][c];
        board[r][c] = '#';

        // Explore all directions
        const found =
            backtrack(r + 1, c, index + 1) ||
            backtrack(r - 1, c, index + 1) ||
            backtrack(r, c + 1, index + 1) ||
            backtrack(r, c - 1, index + 1);

        // Restore (backtrack)
        board[r][c] = temp;

        return found;
    }

    // Try starting from each cell
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (backtrack(r, c, 0)) return true;
        }
    }

    return false;
}

// Time: O(m * n * 4^L) where L = word length
// Space: O(L) for recursion
```

---

25. N-Queens

Problem

Place n queens on an n x n board such that no two queens attack each other.

```
function solveNQueens(n) {
    const result = [];
    const board = Array.from({ length: n }, () => '.'.repeat(n));

    const cols = new Set();
    const diag1 = new Set();  // row - col
    const diag2 = new Set();  // row + col

    function backtrack(row) {
        if (row === n) {
            result.push([...board]);
            return;
        }

        for (let col = 0; col < n; col++) {
            // Check if position is safe
            if (cols.has(col) || diag1.has(row - col) || diag2.has(row + col)) {
                continue;
            }

            // Place queen
            cols.add(col);
            diag1.add(row - col);
            diag2.add(row + col);
            board[row] = board[row].substring(0, col) + 'Q' + board[row].substring(col + 1);

            backtrack(row + 1);

            // Remove queen (backtrack)
            cols.delete(col);
            diag1.delete(row - col);
            diag2.delete(row + col);
            board[row] = board[row].substring(0, col) + '.' + board[row].substring(col + 1);
        }
    }

    backtrack(0);
    return result;
}

// Time: O(n!), Space: O(n)
```

---

26. Sudoku Solver

```
function solveSudoku(board) {
    solve(board);
}
```

```javascript
function solve(board) {
    for (let r = 0; r < 9; r++) {
        for (let c = 0; c < 9; c++) {
            if (board[r][c] === '.') {
                for (let num = 1; num <= 9; num++) {
                    const char = num.toString();
                    if (isValid(board, r, c, char)) {
                        board[r][c] = char;

                        if (solve(board)) {
                            return true;
                        }

                        board[r][c] = '.';  // Backtrack
                    }
                }
                return false;  // No valid number found
            }
        }
    }
    return true;  // All cells filled
}

function isValid(board, row, col, char) {
    // Check row
    for (let c = 0; c < 9; c++) {
        if (board[row][c] === char) return false;
    }

    // Check column
    for (let r = 0; r < 9; r++) {
        if (board[r][col] === char) return false;
    }

    // Check 3x3 box
    const boxRow = Math.floor(row / 3) * 3;
    const boxCol = Math.floor(col / 3) * 3;
    for (let r = boxRow; r < boxRow + 3; r++) {
        for (let c = boxCol; c < boxCol + 3; c++) {
            if (board[r][c] === char) return false;
        }
    }

    return true;
}

// Time: O(9^(empty cells)), Space: O(1)
```

Summary: DP & Backtracking Patterns

DP Patterns

Pattern	When to Use	Examples
1D DP	Single sequence/array	Climbing Stairs, House Robber
2D DP	Two sequences/grid	LCS, Edit Distance, Unique Paths
Knapsack	Selection with capacity	0/1 Knapsack, Coin Change
State Machine	Multiple states	Stock with Cooldown
Interval DP	Range-based problems	Burst Balloons

Backtracking Patterns

Pattern	When to Use	Examples
Subsets	All possible selections	Subsets, Combination Sum
Permutations	All orderings	Permutations, N-Queens
Grid Search	Path finding	Word Search, Sudoku
Pruning	Early termination	N-Queens diagonal check

Key Differences

- **DP:** Overlapping subproblems, optimal substructure, counting/optimization
- **Backtracking:** Explore all possibilities, constraint satisfaction, find all solutions

---

Continue to Part 5: Binary Search, Heaps & More Solutions